

15.1. La fonction `semer()` — le départ du hasard

`#include <semer>` : `semer(n)` choisit le point de départ des tirages de `hasard()`.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `semer(n)` donne au tirage un **point de départ** choisi. C'est utile pour mélanger avec un nombre qui vient du joueur : le moment où il appuie sur START, par exemple.

La ligne à écrire : `#include <semer>`.

Son argument : un nombre de 0 à 255.

Ce qu'elle coûte : une dizaine d'octets.

Essaie : `semer(images());` au moment où l'on appuie sur START.

On retrouve `semer` dans 1 leçon

[16](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **419 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	61 o	← la console, à l'allumage – la mise en route de la console
main()	31 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre – attend de pouvoir écrire une case
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Hasard	25 o	← hasard() – tire un nombre au hasard
Tuiles	160 o	← CopierTuiles – 10 lettres sur 43

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>nombre()</code>	× 1	10 o sur place, dans <code>main()</code> + <code>EcrireNombre</code> (33 o)
<code>semer()</code>	× 1	9 o sur place, dans <code>main()</code>
<code>hasard()</code>	× 1	3 o sur place, dans <code>main()</code> + <code>Hasard</code> (25 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <semer> : le départ du hasard ----  
  
#include <semer>      // semer() : le départ des tirages  
#include <hasard>  
#include <nombre>
```

2. UNE VARIABLE — TIRAGE

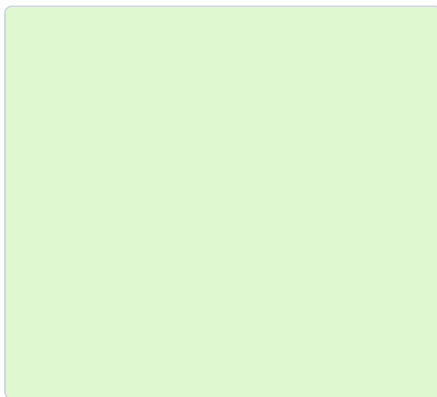
```
uint8_t tirage = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    semer(42);          // le point de départ du hasard  
    tirage = hasard();  // le premier tirage, à partir de là  
    nombre(8, 8, tirage);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un nombre tiré au hasard, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ un tirage est écrit (« 207 »)

✓ c'est bien la valeur de tirage

Ensuite : **16. Un dé, et le hasard du matériel** — chapitre 3 : Retenir, et réagir. · gameboy3 — écrire une cartouche Game Boy en C++